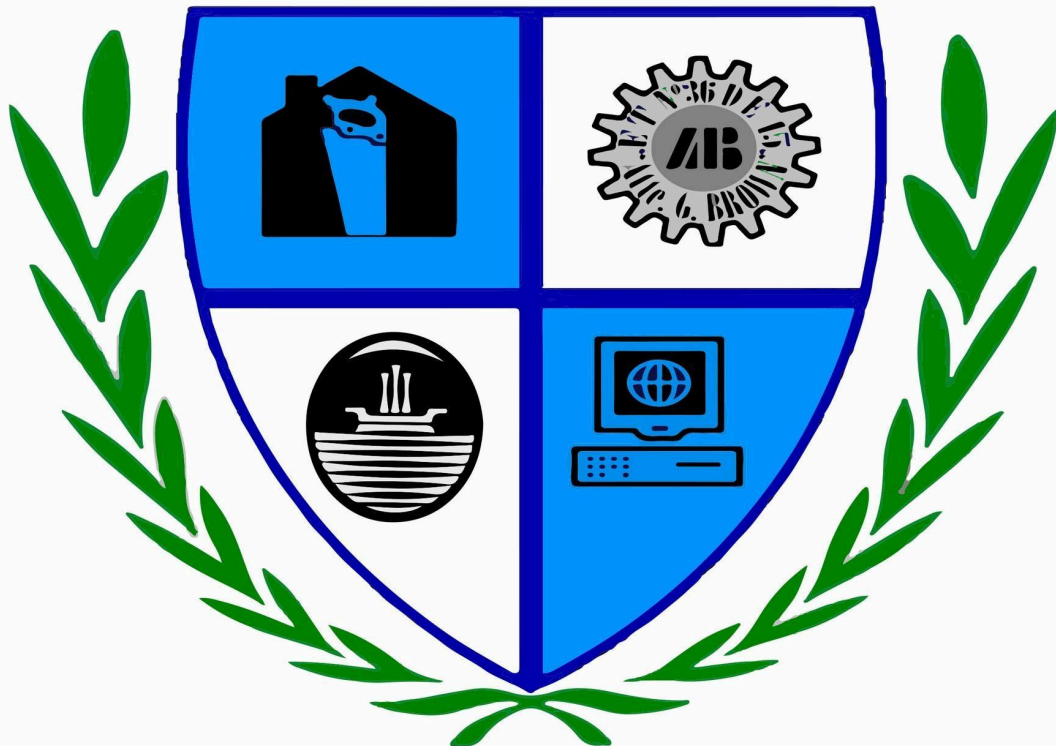


Alte. Guillermo Brown



E.T. N°36 - D.E.15

Nombre de la institución: Alte. Guillermo Brown

Materia: Redes

Docente: Oscar A Obregón

Título: Documentación de desarrollo

Hecho por: Miguel Gonzalez Liptak

Curso: 5°1°

Introducción

El siguiente documento tiene el objetivo de contar cómo está compuesto el escáner de red

Se abordarán los siguientes temas:

- Para qué sirve el programa
- Cómo está armado el sistema (con diagramas)
- Qué métodos se usaron y por qué
- Por qué se eligieron ciertas tecnologías
- Qué problemas aparecen y cómo se solucionaron
- Qué se podría mejorar en el futuro

Para qué sirve el programa

El escáner de red es una aplicación desarrollada en java que se encarga de realizar un escaneo detectar los dispositivos que están conectados a un rango específico de direcciones IPs en una red dejando ver cuáles están activos, el nombre del dispositivo y ofrecer otras funcionalidades

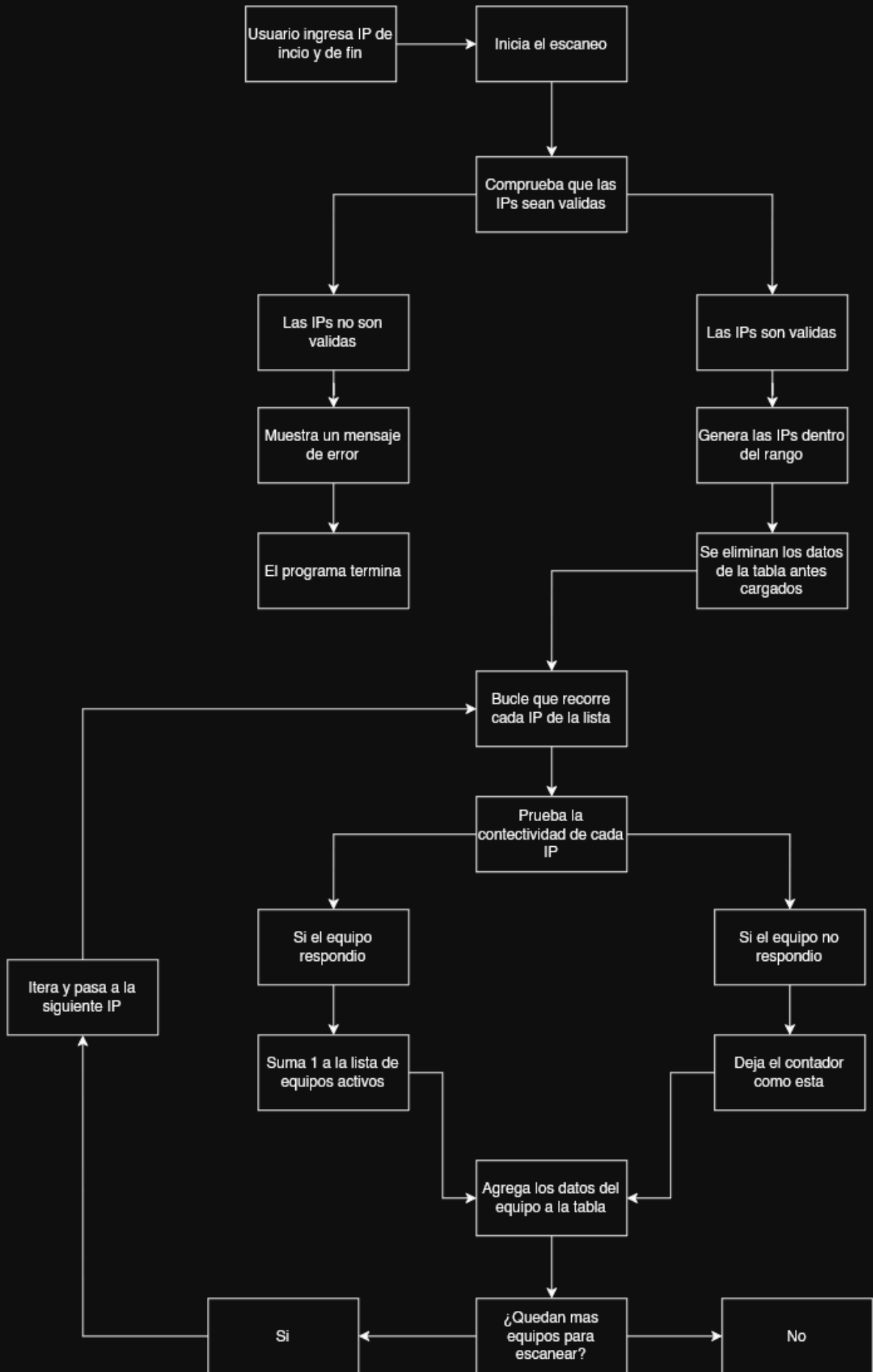
Como esta armado el sistema

El sistema sigue una estructura Modelo-Vista-Controlador para separar la lógica de la parte visual

El modelo define y guarda todos los datos que se van a utilizar para realizar el escaneo como las IPs recibidas, el nombre de los equipos, su estado y el tiempo de respuesta

El controlador se encarga de toda la lógica del escaneo encargado de validar las IPs introducidas por el usuario, transformarlas a número para poder trabajar con ellas, devolverlas a su formato original para mostrarlas al usuario, generar las IPs que se encuentran dentro de un rango y realizar el escaneo

La vista se encarga de toda la parte visual e interactiva. Se comunica con el controlador para enviarle los datos ingresados por el usuario y mostrar los resultados dentro de la tabla.



Qué métodos se usaron y por qué

En la clase controlador se utilizaron varios métodos para realizar el escaneo:

esIpValida encargado de verificar que las IPs introducidas por el usuario sean válidas devolviendo error si el usuario no ingresa nada

```
if (ip == null || ip.isEmpty()) {  
    return false;  
}
```

Que no se pasen de 4 bloques numéricos

```
String[] partes = ip.split("\\.");  
if (partes.length != 4) {  
    return false;  
}
```

Y que estén dentro del rango numérico aceptado para una IP

```
for (String parte : partes) {  
    try {  
        int numero = Integer.parseInt(parte);  
        if (numero < 0 || numero > 255) {  
            return false;  
        }  
    } catch (NumberFormatException e) {  
        return false;  
    }  
}
```

Luego un método para poder transformar las IPs a número utilizando un proceso matemático

```
private long ipANumero(String ip) {  
    String[] partes = ip.split("\\.");  
  
    long n1 = Integer.parseInt(partes[0]);  
    long n2 = Integer.parseInt(partes[1]);  
    long n3 = Integer.parseInt(partes[2]);  
    long n4 = Integer.parseInt(partes[3]);  
    return (n1 * 256 * 256 * 256) + (n2 * 256 * 256) + (n3 * 256) +  
n4;  
}
```

Para pasar cada parte de la IP a decimal se multiplican por 256 siendo esta la cantidad de dígitos que pueden entrar (de 0 a 255) , cada número se multiplica por 256 la cantidad de

espacios que tengan a la derecha, así se obtiene un número que se puede utilizar fácilmente para comparar inicio y fin o bucles

Para transformar devuelta ese número a IP se hace el mismo proceso matemático pero a la inversa

```
private String numeroAIp(long numero) {  
    long parte1 = (numero / (256 * 256 * 256)) % 256;  
    long parte2 = (numero / (256 * 256)) % 256;  
    long parte3 = (numero / 256) % 256;  
    long parte4 = numero % 256;  
    return parte1 + "." + parte2 + "." + parte3 + "." + parte4;  
}
```

Se divide cada parte del número por la cantidad de veces que se multiplicó y el "% 256" se encarga de que el resultado no se pase de 255

Para generar las IPs que están dentro del rango introducido por el usuario

```
public List<String> generarRangoIps(String ipInicio, String ipFin) {  
    List<String> listaIps = new ArrayList<>();  
    long inicio = ipANumero(ipInicio);  
    long fin = ipANumero(ipFin);  
    if (inicio <= fin) {  
        for (long i = inicio; i <= fin; i++) {  
            listaIps.add(numeroAIp(i));  
        }  
    }  
    return listaIps;  
}
```

Crea una lista, utiliza el método para pasar las IPs introducidas por el usuario a número, recorre cada número que haya entre ellos y los transforma a IP utilizando el otro método antes definido y añadiéndolos a la lista

Por último el método utilizado para escanear las IPs

```
public Equipo escanearIp(String ip, int timeout) {  
    String nombre = "Desconocido";  
    boolean respondio = false;  
    long tiempo = 0;  
  
    try {  
        InetAddress address = InetAddress.getByName(ip);  

```

```
    respondio = address.isReachable(timeout);

    if (respondio) {
        nombre = address.getHostName();
        tiempo = timeout;
    }
} catch (Exception e) {
    respondio = false;
    nombre = "Desconocido";
    tiempo = 0;
}

return new Equipo(ip, nombre, respondio, tiempo);
```

Se definen variables por defecto en caso de que el equipo no responda o falle, luego convierte la ip a un objeto de red que java puede leer y se intenta comunicar con el dispositivo dentro del tiempo límite, si responde devuelve true y si responde true intenta conseguir el nombre del dispositivo.

Por ultimo devuelve un objeto para agregar a la lista

Por qué se eligieron ciertas tecnologías

Se eligió programar en Java utilizando visual studio code debido a estar familiarizado con el lenguaje y el programa permitiendo utilizar distintas librerías de java como “java.net” para manejar redes y “javax.swing” para la interfaz gráfica

Qué problemas aparecieron y cómo se solucionaron

Si bien durante el desarrollo no hubieron problemas críticos, se presentaron problemas de lógica, de no saber cómo desarrollar los requerimientos del programa y encontrar que a veces las soluciones requerían de cosas que no conocíamos del todo o no entendíamos, se buscaron maneras de hacer lo mismo de otra forma utilizando elementos que ya conocíamos o conocer estas nuevas funcionalidades

Qué se podría mejorar en el futuro

Hay funcionalidades que no funcionan a pesar de tener los botones para activarlas como guardar los resultados o elegir visualizar solamente los equipos activos